

• altri esempi in cui si decide quale segno vogliamo dell'equazione

[80]: let sol a b c =

let delta = b * b - 4 * a * c

in

let sol' op = // sol' viene utilizzato per calcolare entrambi le soluzioni

(op (-b) (sqrt delta)) / (2 * a

in

(sol' (+.), sol' (-.));;

[80]: val sol: float -> float -> float -> float * float = <fun>

[81]: sol 1. (-.5.) 6.;

[81]: - float * float = (3., 2.)

Funzioni Curried

I nomi delle funzioni e i vari parametri sono separati da spazi (senza parentesi o virgole)

Le funzioni definite usando la notazione con gli spazi si dicono curried ma in realtà nessuno vieta di usare parentesi e virgole:

[41]: let soma (x, y) = x + y

soma (3, 4)

[41]: val soma: int * int -> int = <fun>

-: int = 7

• ora così stiamo scrivendo una funzione con un unico parametro int * int

- la funzione monadico una coppia che viene destrutturata nelle variabili x e y

Applicazione parziale di funzioni

- la funzione curried consente di applicare una funzione parzialmente passando solo una parte dei parametri

esempio

let somma $x\ y = x + y$

val somma: $int \rightarrow int \rightarrow int$

somma 10 // PASSAHO un solo argomento e somma

$\therefore int \rightarrow int = \langle fun \rangle$ // ritorna una funzione a cui va passato ancora un parametro

Controllo del flusso dell'esecuzione in Ocaml

- In paradigma imperativo sono:

- if

- while

Invece nel paradigma funzionale abbiamo

- un'espressione di scelta condizionale: f

- ricorsione

espressioni scelte condizionale if

deve sempre avere due rami: then e else

- 2 rami dovranno contenere a loro volta espressioni dello stesso tipo
- l'if potrà essere usato "a destra" di una dichiarazione

```
[47]: let x = 23;  
      let y = if x > 10 then 1 else 0
```

```
[47]: val x: int = 23  
      val y: int = 1
```

- è molto simile all'operatore di scelta condizionale

(_ ? _ : _)

esempio JAVASCRIPT: var y = (x > 10 ? 1 : -1)

Ricorsione

- PER definire funzioni ricorsive è necessario utilizzare il costrutto **let rec**

```
[48]: let fact m =  
      if m <= 0 then 1 else m * fact(m-1) // DA errore perché non abbiamo  
                                         utilizzato il costrutto rec
```

```
[49]: let rec fact m =  
      if m <= 0 then 1 else m * fact(m-1)
```

```
[49]: val fact: int -> int = <fun>
```

$$\text{fact}(m) = \begin{cases} 1 & \text{se } m \leq 0 \\ m \cdot \text{fact}(m-1) & \text{altrimenti} \end{cases}$$

- I costrutti if e ricorsione rende le definizioni di funzioni in OCaml concettualmente molto simili a come le funzioni sarebbero definite in termini matematici

Funzioni Mutualmente Ricorsive (con omd)

- Ogni funzione può richiamare solo funzioni definite precedentemente

Funzioni Mutualmente ricorsive vanno definite una dopo l'altra separate da omd

```
[1]: let rec PARI m =  
  match m with  
  | 0 -> true  
  | x -> DISPARI (x-1)
```

Match espressione with

ESEMPIO

0 -> true // se espressione = 0 restituisce true

x -> DISPARI (x-1) // altrimenti prende l'espressione
come argomento e chiama
DISPARI

```
and dispari m =  
  match m with  
  | 0 -> false  
  | x -> pari (x-1);
```

```
[1]: val pari: int -> bool = <fun>  
     val dispari: int -> bool = <fun>
```

se chiamiamo per esempio

• $\text{PARI } 4 \rightarrow \text{DISPARI } (3)$

$\text{DISPARI } (3) \rightarrow \text{PARI } (2)$

$\text{PARI } (2) \rightarrow \text{DISPARI } (1)$

$\text{PARI } (0) \rightarrow \text{true}$

$\text{PARI } (4) \rightarrow \text{true}$

• $\text{PARI } (3) \rightarrow \text{DISPARI } (2)$

$\text{DISPARI } (2) \rightarrow \text{PARI } (1)$

$\text{PARI } (1) \rightarrow \text{DISPARI } (0)$

$\text{DISPARI } (0) \rightarrow \text{true}$

type inference

- Abbiamo già visto che il toplevel di Ocaml ci fornisce il tipo di ogni espressione che inseriamo

Il tipo di un'espressione viene inferito analizzando

- i valori inseriti
- gli operatori usati
- i valori letterali e gli operatori hanno un loro specifico segno

esempio + deve avere gli operandi in int
+. deve avere gli operandi float

Infatti:

$x + y \rightarrow x \text{ e } y \text{ devono essere int}$

$(x+1)-(x+4)$ // supera il controllo dei tipi perché x viene inferito in int

$(x+1)-(x+4.0)$ // non supera il controllo dei tipi perché x può essere o int o float

type inference funzioni

Si analizzano:

- i valori inseriti.
- gli operatori usati
- gli argomenti / valori di ritorno delle funzioni

vediamo degli esempi

Funzione
PARAMETRI FORMALI
let 1° (sum - IF - TRUE) (test first second) =
2° (IF (test first) then first else 0)
3° + (if test second then second else 0) ;;

GUARDA = QUANTI RIS = BOOLEANO

Funzione

PARAMETRO

Intero se non non abbiamo fare la somma

Int

Int

PER LO STESSO RAGIONAMENTO DI PRIMA

$t \rightarrow F \rightarrow S = \text{ris}$

$(\overset{F}{\text{Int}} \rightarrow \overset{T}{\text{Bool}}) \rightarrow \underset{\text{first}}{\text{Int}} \rightarrow \underset{0}{\text{Int}} \rightarrow \underset{\text{ris}}{\text{Int}} + (\text{Int} \rightarrow \text{Bool}) \rightarrow \text{Int} \rightarrow \text{Int} \rightarrow \text{Int}$

In questo caso

test = predicato ritorna un Booleano (In questo caso: se pari true se non false)

Funzioni polimorfe

- una funzione polimorfa si può applicare ad argomenti di tipo diversi
- il risultato avrà per certo lo stesso tipo dell'argomento passato

esempio

[53]: let id x = x

[53]: val id: 'a -> 'a = <fun>

- il ">" è un operatore polimorfo funzione con tutti i tipi